Rumos



Java Backend com Spring Boot

Novembro de 2026

Siga-nos —

C2 General

Agenda da Formação

01. Spring Boot & Rest Básico

02. API Design, DTOs & Validação

03. Persistência com JPA & Hibernate

04. Integração Externa (TMDB)

05. Qualidade, Erros & Testes

06. Spring Security, JWT & OAuth2

07. Empacotamento & Container

08. Kubernetes Local (Minikube)

Rumos

Qualidade:

Erros, Testes e Observabilidade

ERRORS

contrato previsível

TESTS

evidência executável

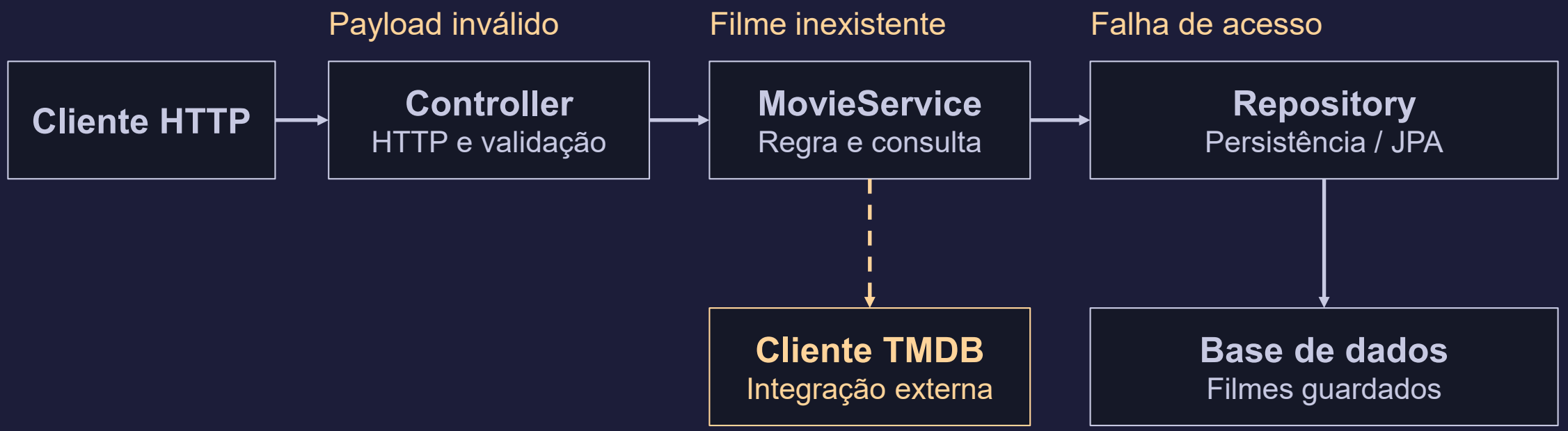
HEALTH

interface operacional

Roteiro da aula

Tema	Objetivo
Contexto e diagnóstico	**Reconhecer falhas no MyMovies**
Contrato de erro e logs	**Separar resposta pública e diagnóstico**
Testes e diagnóstico	Escolher o teste e ler uma falha
Configuração e health	Interpretar o sinal operacional
Laboratório	Consolidar conhecimento e evoluir API

MyMovies: fronteiras e pontos de falha



Que falha chega ao cliente? Que evidência fica para o diagnóstico?

PROBLEM

Funciona. Mas é previsível quando falha?

404

Recurso inexistente pode produzir respostas diferentes dependendo de onde falha.

400

Validation precisa de um contrato compreensível para o consumidor.

5xx / remoto

Falhas de dependência não devem vazar stack trace ou detalhes internos.

Qualidade de API inclui o comportamento de falha, não apenas o happy path.

Atividade : que resposta espera o cliente?

A GET de um filme que não existe

B POST com um campo obrigatório vazio

C A dependência TMDB não responde

Resposta: status proposto, code estável e uma mensagem pública segura.

Extensão: que decisão adicional exige a falha remota?

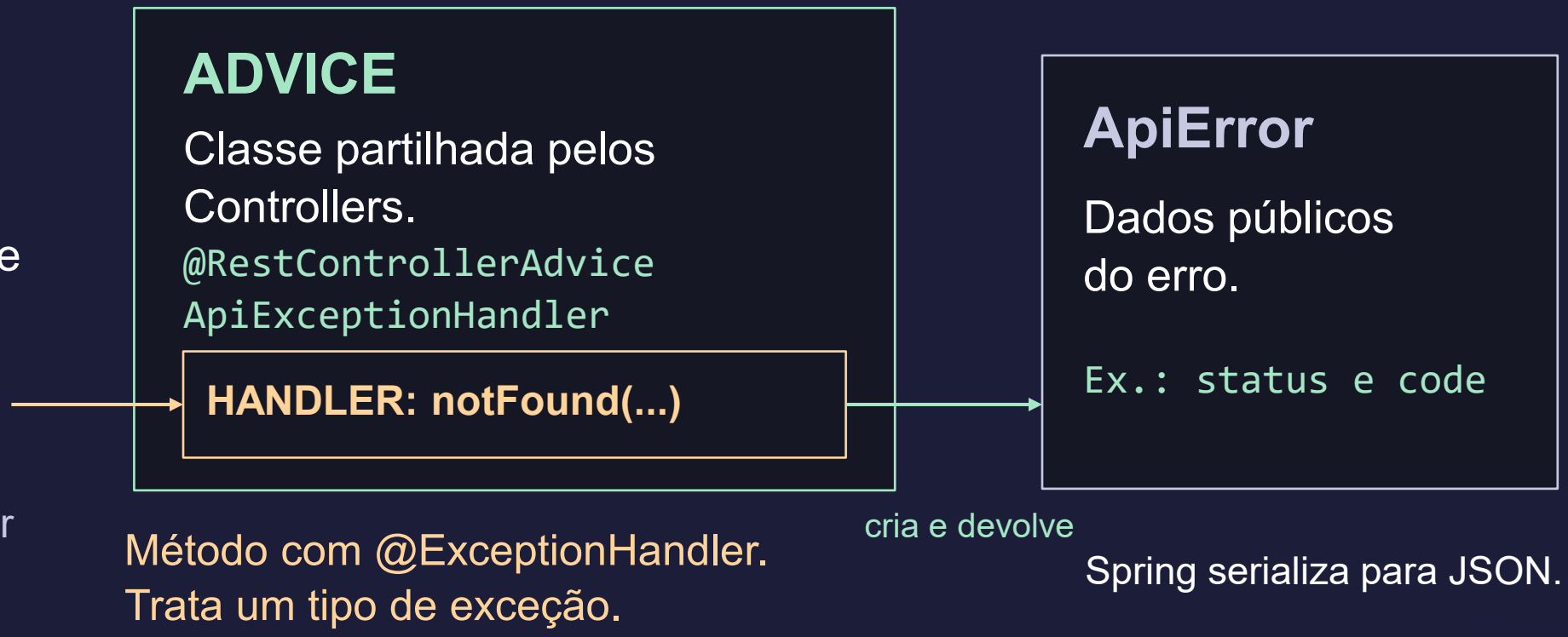
Um error contract reduz surpresa para o cliente

```
1 {
2   "timestamp": "...",
3   "status": 404,
4   "code": "MOVIE_NOT_FOUND",
5   "message": "Movie not found: 99",
6   "path": "/api/movies/99"
7 }
```

- **status**: semântica HTTP
- **code**: identificador estável da categoria
- **message**: informação pública útil
- **path**: contexto da request
- sem stack trace no body

Advice, Handler e ApiError: quem faz o quê?

O Spring MVC recebe a exceção que escapou do Controller e procura o método que a trata.



Exceções e advice na estrutura do projeto

Proposta para src/main/java/example/mymovies

```
MyMoviesApplication.java
model/
  Movie.java
exception/
  MovieNotFoundException.java
web/error/
  ApiExceptionHandler.java
  ApiError.java
```

Raiz: @SpringBootApplication inclui o component scan.

movie: domínio e caso de uso.

web.error: tradução para HTTP. Advice e ApiError ficam aqui.

Cada classe pública tem ficheiro.

O nome das pastas é uma escolha. O package raiz da aplicação define o scan padrão.

MovieNotFoundException: uma classe Java própria

Ficheiro: movie/MovieNotFoundException.java

```
package pt.rumos.mymovies.exception;
public class MovieNotFoundException extends RuntimeException {

  public MovieNotFoundException(Long id) {
    super("Movie not found: " + id);
  }
}
```

O Service cria a instância com new quando o filme está ausente.
Esta classe dispensa @Component e @Autowired.

O tipo da exceção permite ao advice escolher o handler. O HTTP 404 vem da camada web.

Filme ausente: a exceção nasce no MovieService

MovieService.java

```
@Service
public class MovieService {
    //Declaração e injeção do repository…

    public MovieResponse findById(Long id) {
        Movie movie = repository.findById(id) .orElseThrow(() ->
                                                new MovieNotFoundException(id));

        return toResponse(movie);
    }
}
```

O papel de cada componente

Repository: devolve Optional.

Service: decide que a ausência impede o caso de uso.

new cria a exceção apenas quando falta o filme.

GET /api/movies/99 chama service.get(99L). Sem filme, a exceção sobe até à camada web.
No próximo passo, o advice transforma essa exceção em HTTP 404.

HTTP 404: implementação do ApiExceptionHandler

ApiExceptionHandler.java

```java
@RestControllerAdvice
public class ApiExceptionHandler {

  @ExceptionHandler(MovieNotFoundException.class)
  @ResponseStatus(HttpStatus.NOT_FOUND)
  public ApiError notFound(MovieNotFoundException ex,
                           HttpServletRequest request) {

      //Cria uma instância de ApiError e o retorna
  }
}
```

ApiError.java

```java
public record ApiError(
    Instant timestamp,
    int status,
    String code,
    String message,
    String path) {}
```

@ResponseStatus define o HTTP. O campo status descreve o body. Ambos devem indicar 404.

Como o Spring encontra e aplica o advice

No arranque: o component scan regista o advice como bean

```
@RestControllerAdvice
public class ApiExceptionHandler { ... }
```

No pedido: a exceção escapa do Controller e o Spring procura um handler compatível.



@ExceptionHandler escolhe o tipo. @ResponseStatus define o HTTP.
@RestControllerAdvice escreve o retorno no body da resposta.

HTTP 400: @Valid liga o payload ao contrato

DTO e Controller

```
record MovieRequest(
    @NotBlank String title) {}
--------------------
@PostMapping
public Movie create(
    @Valid @RequestBody
         MovieRequest input) {

  return service.create(input);
}
```

No mesmo advice

```
@ExceptionHandler( MethodArgumentNotValidException.class)
@ResponseStatus(HttpStatus.BAD_REQUEST)
public ApiError invalid(HttpServletRequest request) {

   return new ApiError(Instant.now(), 400,
      "VALIDATION_ERROR",
      "Invalid request body",
      request.getRequestURI());
}
```

```
POST /api/movies
{"title":""}
```

```
HTTP 400
code: VALIDATION_ERROR
```

Rumos

HTTP 500: tratamento da falha inesperada

No ApiExceptionHandler, além dos métodos 404 e 400

```
@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ApiError unexpected(Exception ex, HttpServletRequest request) {
  return new ApiError(Instant.now(), 500, "INTERNAL_ERROR",
      "Unexpected error", request.getRequestURI());
}
```

Resultado para o cliente: HTTP 500 e mensagem genérica

O advice mantém os handlers específicos de 404 e 400.
A mensagem pública protege os detalhes internos da falha.

Logging: registos do que acontece na aplicação

Logging regista eventos durante a execução para ajudar a compreender o comportamento da aplicação.

Tratamento de exceções

Decide como lidar com a falha.
No advice, define a resposta HTTP.

Exemplo: filme ausente recebe 404.

Registo de eventos

Descreve o que aconteceu em execução.
Também regista operações normais.

O Logger é o objeto usado no código.

Exemplo de entrada de log, formato simplificado e ilustrativo

```
INFO MovieLogDemo : event=MOVIE_LOOKUP id=99
```

Dependência de logging no pom.xml

Com spring-boot-starter-web no Boot 3

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Com este starter, não adicionar logging de novo.
Sem um starter que o inclua, usar spring-boot-starter-logging.

SLF4J

API no código Java.
Logger e LoggerFactory.

Logback

Implementação padrão.
Escreve os eventos.

Spring Boot

Configura níveis e destinos.
Consola por padrão.

Primeiro Logger, sem tratamento de exceções

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class MovieLogDemo {
  private static final Logger log = LoggerFactory.getLogger(MovieLogDemo.class);

  public void observar(long id) {
    log.info("event=MOVIE_LOOKUP id={}", id);
    log.debug("event=LOOKUP_DETAIL id={}", id);
  }
}
```

Níveis de log: importância e detalhe

Nível	Uso sugerido para os cenários de MyMovies
TRACE	Detalhe muito fino de uma sequência de execução.
DEBUG	Informação para investigar. Ex.: id pesquisado.
INFO	Evento normal relevante. Ex.: aplicação iniciou.
WARN	Situação merece atenção. Ex.: retry por timeout.
ERROR	Operação falhou. Ex.: falha inesperada no GET.

O nível acompanha o impacto do evento, não o status HTTP automaticamente.

SLF4J tem estes cinco níveis. OFF é uma configuração para desativar.

Configuração: quais mensagens aparecem?

src/main/resources/application.properties

```
logging.level.root=INFO
logging.level.example.mymovies=DEBUG
```

No package configurado

DEBUG a ERROR passam.
TRACE fica desativado.

Restantes loggers: root=INFO.

Logging no handler: acrescentar o registo do erro

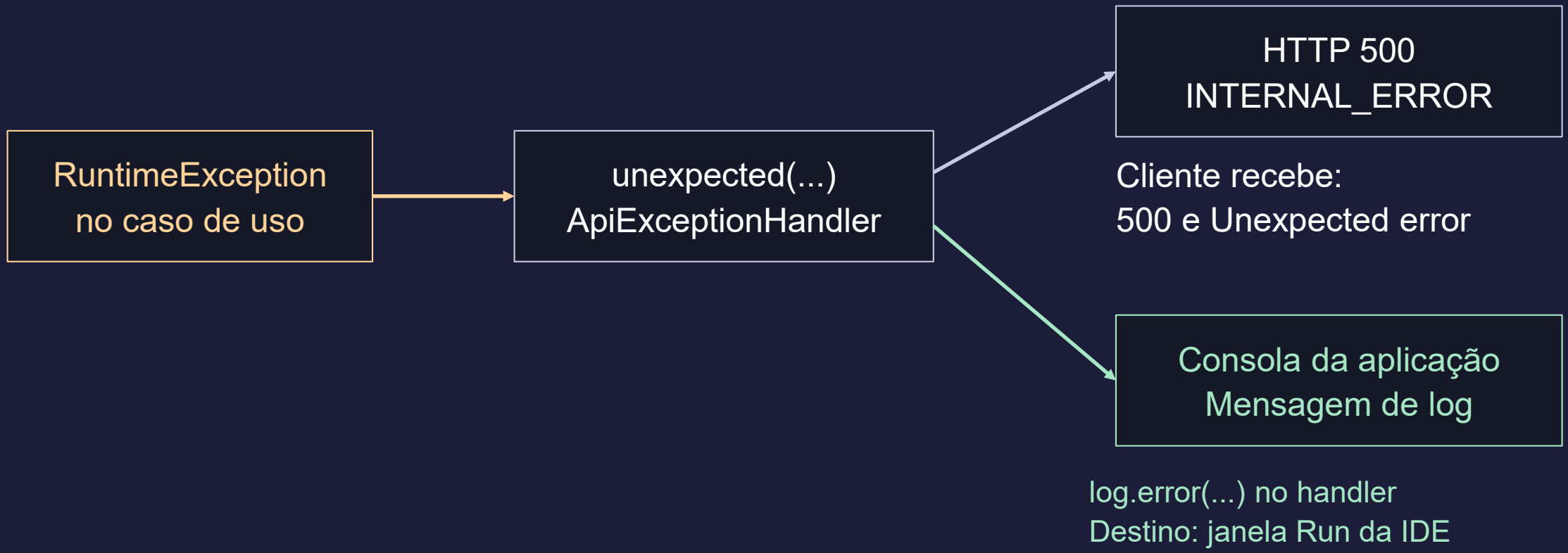
No ApiExceptionHandler, declarar o campo da classe

```
private static final Logger log =
                    LoggerFactory.getLogger(ApiExceptionHandler.class);
```

Dentro de unexpected(...), antes do return do ApiError

```
log.error("event=API_UNEXPECTED path={} type={}",
        request.getRequestURI(), ex.getClass().getSimpleName());
```

Uma falha, duas saídas observáveis



Roteiro da aula

Tema	Objetivo
Contexto e diagnóstico	Reconhecer falhas no MyMovies
Contrato de erro e logs	Separar resposta pública e diagnóstico
Testes e diagnóstico	**Escolher o teste e ler uma falha**
Configuração e health	Interpretar o sinal operacional
Laboratório	Consolidar conhecimento e evoluir API

Teste unitário: um comportamento com dados controlados

Um teste unitário executa uma pequena unidade de código e compara o resultado com uma expectativa explícita.

Unidade real
Uma regra ou classe, como MovieService.

Dependências controladas
Sem base de dados nem TMDB real.
Um mock é opcional.

Num projeto Spring Boot
JUnit Jupiter: @Test e asserções.
Mockito: colaboradores controlados.

Unitário: objetos criados com new.
Sem ApplicationContext.
@SpringBootTest carrega contexto.

A prova cobre regras Java. Wiring, proxies e HTTP exigem outras fronteiras de teste.

Pastas e dependência de teste no projeto

```
src/
  main/
    java/example/mymovies/
      MovieTitle.java
    resources/
      application.yml
  test/
    java/example/mymovies/
      MovieTitleTest.java
    resources/  (dados de teste)
```

pom.xml, dentro de dependencies

```
<dependency>
  <groupId>
    org.springframework.boot
  </groupId>
  <artifactId>
    spring-boot-starter-test
  </artifactId>
  <scope>test</scope>
</dependency>
```

main contém a aplicação. test contém testes e dados exclusivos dos testes.
O nome MovieTitleTest ajuda o Maven a descobrir a classe.

Primeiro teste: limpeza do título sem mocks

src/main/java: MovieTitle.java

```
package example.mymovies;

class MovieTitle {
  String clean(String title) {
    return title.trim();
  }
}
```

src/test/java: MovieTitleTest.java

```
package example.mymovies;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class MovieTitleTest {
  @Test
  void removesEdgeSpaces() {
    var rule = new MovieTitle();
    var actual = rule.clean("  Alien  ");
    assertEquals("Alien", actual);
  }
}
```

assertEquals compara valor esperado e resultado real.
Aqui, clean remove espaços das extremidades.

Execução e sinais de um teste útil

Na IDE
Run junto de @Test ou de MovieTitleTest.

No terminal, na pasta do pom.xml
mvn -Dtest=MovieTitleTest test

Suite do projeto
mvn test

Arrange: criar regra e dados conhecidos
Act: chamar clean(" Alien ")
Assert: esperar "Alien"

- Nome descreve o comportamento.
- Asserção verifica um resultado útil.
- Cada teste é independente.
- Dados evitam rede, relógio e aleatoriedade.

MovieService: injeção no Spring e controlo no teste

Aplicação: Spring fornece o Repository

```
@Service
public class MovieService {
  private final MovieRepository repository;

  public MovieService(MovieRepository repository) {
    this.repository = repository;
  }
  …
}
```

Teste: nós fornecemos o Repository

```
@Mock
private MovieRepository movieRepository;

@InjectMocks
private MovieService movieService;

when(repository.findById(99L))
            .thenReturn(Optional.empty());
```

Service real executa a regra. O mock controla a ausência, sem SQL.
when(...).thenReturn(...) prepara a resposta do colaborador. A asserção vem no teste seguinte.

MovieServiceTest: a ausência lança a exceção esperada

```
@ExtendWith(MockitoExtension.class)
class MovieServiceTest {
    @Mock private MovieRepository movieRepository;

    @InjectMocks private MovieService movieService;

  @Test
  void absentMovieThrows() {

    when(repository.findById(99L)).thenReturn(Optional.empty());

    assertThrows(MovieNotFoundException.class, () -> service.get(99L));
    verify(movieRepository, times(1)).findById(99L);

  }
}
```

Arrange
Criar Service e mock.
Preparar findById vazio.

Act e Assert
A lambda chama get(99L).
assertThrows verifica o tipo.

HTTP e JSON ficam fora.

A mesma falha pede provas em fronteiras diferentes

Unitário MovieService	MockMvc Controller e advice	Persistência @DataJpaTest
findById vazio	GET /api/movies/99	Repository real
assertThrows(...)	status().isNotFound()	Guardar e consultar filme
Prova a regra de ausência	Prova HTTP e JSON	Prova o acesso aos dados

MockMvc executa o GET no Controller e no advice



Controller e advice reais. MovieService mockado.

O builder standalone regista o advice explicitamente. A request completa usa /api/movies/99.

MovieControllerTest: HTTP 404 e code estável

```
// Inicio e declaração da classe omitidos…

private MockMvc mockMvc;

@Mock private MovieService movieService;
@InjectMocks private MovieController movieController;

@BeforeEach
void setUp() {
   mockMvc = MockMvcBuilders.standaloneSetup(movieController)
                        .setControllerAdvice(new ApiExceptionHandler()).build();
}

@Test
void getById_WhenNotFound_ShouldReturn404AndApiError() throws Exception {

    when(movieService.findById(99L)).thenThrow(new MovieNotFoundException(99L));

    mockMvc.perform(get("/api/movies/99"))
                  .andExpect(status().isNotFound())
                  .andExpect(jsonPath("$.code").value("MOVIE_NOT_FOUND"))
                  .andExpect(jsonPath("$.status").value(404)); } }
```

1 Controller real

2 Advice real

3 Service controlado

4 HTTP e JSON

Sucesso e ausência no mesmo GET

Filme existe

Repository devolve um filme.
service.get(1L) devolve os dados.

No teste web:
status().isOk()
jsonPath("$.id").value(1)

Filme ausente

Repository devolve Optional.empty().
Service lança a exceção.

No teste web:
status().isNotFound()
jsonPath("$.path").value("/api/movies/99")

Observar o MyMovies em execução

A aplicação arrancou. Agora queremos saber como se comporta.

Estado

Consegue verificar a base de dados?

Utilização

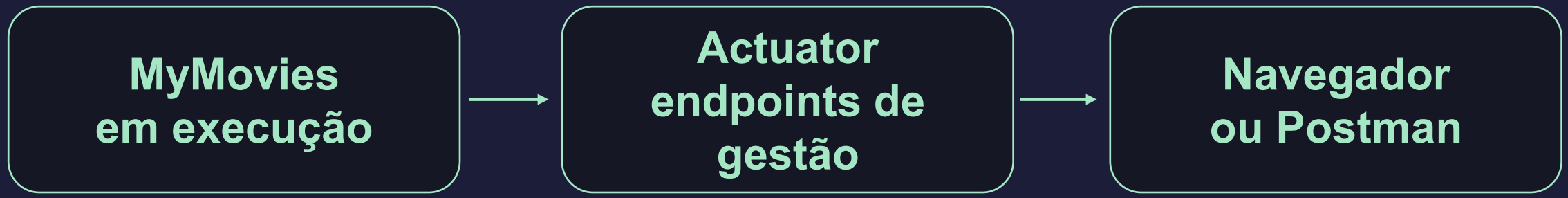
Quanta memória usa? Quantas chamadas recebeu?

Desempenho

Quanto tempo demoram as respostas?

Observabilidade: usar sinais da aplicação para responder a estas perguntas.

Actuator: acesso aos sinais da aplicação



O Spring prepara várias medições automaticamente.

health

Estado dos checks disponíveis

metrics

Valores medidos pela aplicação

Configuração mínima para a verificação local

1 Dependência no pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-actuator
  </artifactId>
</dependency>
```

Exposição apenas de health e metrics:

```
management.endpoints.web.exposure.include=health,metrics
```

3 Reiniciar a aplicação e abrir http://localhost:8080/actuator/health

health: uma leitura simples do estado

```
GET /actuator/health
```

```
{"status":"UP"}
```

Os checks incluídos reportam um estado saudável.

Detalhes na demonstração local

```
management.endpoint.health.show-details=always
```

Quando existe um DataSource, pode aparecer db. Também pode aparecer diskSpace.

Panorama das métricas prontas

O que observamos	Exemplo de nome	O que podemos ler
Memória Java	jvm.memory.used	Memória utilizada
CPU da aplicação	process.cpu.usage	Utilização do processador
Chamadas HTTP	http.server.requests	Quantidade e duração
Ligações à base	jdbc.connections.active	Ligações em uso

A lista varia com o projeto. As métricas HTTP requerem chamadas anteriores.

Tipos de medição: valor, contagem e duração

Tipo	Ideia simples	Exemplo
Gauge **Valor instantâneo**	Pode subir ou descer	Memória utilizada agora
Counter **Contador**	Acumula eventos	Quantidade de um evento
Timer **Temporizador**	Conta eventos e soma o tempo	Chamadas HTTP e duração

http.server.requests é um Timer. A sua resposta inclui COUNT e TOTAL_TIME.

Acesso às métricas no navegador ou Postman

1 Lista de nomes

```
http://localhost:8080/actuator/metrics
```

Exemplo de parte da lista

```
"names": [
  "jvm.memory.used",
  "http.server.requests"
]
```

2 Uma métrica

Acrescentar o nome ao caminho:

/actuator/metrics/jvm.memory.used

As chamadas usam GET. A resposta é JSON, que vamos ler campo a campo.

Uma métrica de memória: nome, unidade e valor

```
GET /actuator/metrics/jvm.memory.used?tag=area:heap
```

heap: memória usada para objetos Java

Exemplo parcial ilustrativo

```
"baseUnit": "bytes",
"measurements": [
  {"statistic": "VALUE",
   "value": 88080384}
]
```

84 MiB

Memória utilizada nesta leitura

?tag=area:heap filtra a leitura. O valor pode mudar na próxima consulta.

Chamadas HTTP: quantidade e tempo

```
GET /actuator/metrics/http.server.requests
```

```
Filtro de uma rota, se presente: ?tag=uri:/api/movies/{id}
```

Campo	Exemplo	Leitura simples
COUNT	10	Dez chamadas observadas
TOTAL_TIME	2 segundos	Tempo somado das chamadas

200 ms

Média do exemplo: 2 s ÷ 10 = 200 ms

OBSERVABILIDADE: CATEGORIAS E TIPOS

[STATUS] Diagnóstico & Saúde

- **health** → Status global da App e base H2
- **info** → Uptime, versão do projeto & JDK
- **startup** → Tempos de arranque por etapa

[METRICS] Métricas & Desempenho

- **metrics** → CPU, Memória Heap & Threads
- **http.server.requests** → Contagem & Latência HTTP
- **prometheus** → Exportador OpenMetrics / Grafana

[INSPECT] Arquitetura & Rotas

- **env** → Propriedades & Segredos mascarados
- **beans** → Grafo de Beans no Spring IoC
- **mappings** → Rotas REST & Controller Endpoints
- **configprops** → Valores de @ConfigurationProperties

[LOGS] Logs & Diagnóstico Avançado

- **loggers** → Alteração de nível de log em Runtime
- **threaddump** → Análise de Threads & Deadlocks
- **heapdump** → Download de .hprof (Memory Leaks)

Checkpoint: aula-05-quality

- ApiError: os mesmos cinco campos
- Advice: GET 404 e POST inválido 400
- ServiceTest: sucesso e ausência
- MockMvc: HTTP e code comprovados
- Health: resposta e alcance explicados
- Log: diagnóstico sem segredos

`checkpoint/aula-05-quality`

Próxima aula: proteger recursos com Spring Security, JWT e OAuth2 Resource Server.

As interfaces concretas de qualidade do MyMovies

CLIENTE API

GET /api/movies/99
HTTP 404
MOVIE_NOT_FOUND

DESENVOLVEDOR

MovieServiceTest
MovieControllerTest
Log API_UNEXPECTED

PLATAFORMA

GET /actuator/health
Indicadores incluídos
Limites documentados

A resposta trata a falha, o teste comprova o contrato e o health observa o ambiente configurado.